Group X Progress Report:
My Group's Project Name

First Author, Second Author, Third Author, Fourth Author
{ravicv3,macid2,macid3,macid4}@mcmaster.ca

1 Introduction

Phishing emails remain a prevalent and costly cybersecurity threat. They cost organizations billions of dollars annually and frequently lead to large-scale data breaches and identity theft (Almomani et al., 2013). These fraudulent messages are designed to deceive recipients into giving up sensitive information, such as passwords, bank credentials, or personal data, often by mimicking legitimate communications from trusted entities. While modern spam filters successfully intercept many obvious threats, sophisticated phishing attempts often closely imitate the language and formatting of genuine emails, allowing them to evade traditional rule-based detection systems.

The goal of this project is to develop a Natural Language Processing (NLP) classifier capable of accurately distinguishing phishing emails from legitimate ones. By framing the problem as a binary text classification task, an additional layer of automated protection can be developed to help reduce the success rate of phishing attacks. Several challenges inherently make this task difficult: phishing emails often employ language that closely mirrors legitimate business correspondence, the linguistic strategies used by attackers evolve continuously to evade existing filters, and class imbalance is a recurring concern because phishing emails are typically far less common than legitimate emails in real-world environments.

To address these challenges, a balanced dataset was curated from well-known public email corpora. This dataset was used to explore both traditional machine learning approaches, such as Bag-of-Words with Logistic Regression, and modern transformer-based models. The classifiers were evaluated using the F1-score as the primary metric, supplemented by accuracy, precision, and recall, to ensure a robust assessment of phishing detection performance.

2 Related Work

Phishing email detection has been studied extensively in both the cybersecurity and NLP communities. Almomani et al. (2013) provide a comprehensive survey of phishing email filtering techniques, categorizing approaches into list-based, heuristic, visual-similarity, and machine-learning methods. They highlight that content-based machine learning classifiers consistently outperform static rule sets as phishing tactics evolve.

Early machine learning work by Fette et al. (2007) demonstrated that a Random Forest classifier trained on hand-crafted features, including the presence of URLs, the structure of embedded links, and the use of JavaScript, could identify phishing emails with over 96% accuracy. In a complementary study, Abu-Nimeh et al. (2007) compared Logistic Regression, Support Vector Machines, Bayesian Additive Regression Trees, and Neural Networks on a bag-of-words representation of email text. They found that no single algorithm dominated across all evaluation metrics and that the quality of the feature representation was at least as important as the model selection itself.

Basnet et al. (2008) explored an expanded feature set that combined textual cues with structural email properties, such as sender domain and header anomalies. This approach achieved strong results with an SVM classifier and reinforced the value of metadata-aware models.

More recently, deep learning techniques have advanced the state of the art. Alhogail and Alsabih (2021) proposed a Graph Convolutional Network applied to email body text, reaching approximately 98% accuracy and showing that representation-learning methods can capture subtle semantic patterns that traditional feature engineering misses. The success of pre-trained language models such as BERT (Devlin et al., 2019) has significantly impacted the field. Fine-tuned transformer models have been shown to exceed a 95% F1-score on phishing benchmarks by leveraging contextual word embeddings that capture the nuanced phrasing characteristic of social-engineering attacks.

This project draws on insights from both traditional and neural approaches. The curated phishing email corpus released by Champa et al. (2024) was utilized, which aggregates data from the TREC,

SpamAssassin, and CEAS corpora. The subsequent experiments evaluate Bag-of-Words baselines alongside fine-tuned transformer architectures to identify the most effective strategy for the balanced dataset.

3 Dataset

3.1 Overview

The dataset used for this project is a curated subset of the Figshare "Seven Phishing Email Datasets" (Champa et al., 2024), a large repository of publicly available email corpora compiled for phishing detection research. This subset draws from five individual datasets within the repository: TREC-5, TREC-6, TREC-7, CEAS-08, and SpamAssassin. These sources were selected because they contain rich email metadata, including sender address, receiver address, subject line, and a binary URL flag, in addition to the raw email body, providing a realistic representation of the features that real-world email filters would evaluate.

3.2 Data Collection and Balancing

To construct a balanced dataset, 1,500 phishing and 1,500 legitimate entries were extracted from each of the five sub-datasets, yielding a total of 15,000 data points with a perfect 50/50 class distribution. This design prevents any single source corpus from dominating the training signal and ensures sufficient data points from both classes. During collection, rows with missing or incomplete fields were skipped; because the original repository is sufficiently large, omitting these entries did not compromise the target sample size. Data collection proceeded by filtering on the provided label column and selecting entries sequentially until 1,500 valid data points per class were obtained from each sub-dataset. The final dataset was assembled on February 19, 2026.

3.3 Dataset Format

The data is stored as a single CSV file in which each row represents one email. The columns are summarized in Table 1.

Column	Description	Type
Sender	Email address of the sender	Text
Receiver	Email address of the recipient	Text
Date	Timestamp of the email	Text
Subject	Subject line text	Text
Body	Body content text	Text
Label	Ground-truth class	Binary
Urls	Whether the email contains URLs	Binary

Table 1: Dataset schema. The `Label` column is coded as 0 (legitimate) or 1 (phishing); the `Urls` column is coded as 0 (no URLs) or 1 (contains URLs).

3.4 Preprocessing

Minimal preprocessing was applied to the raw data. HTML tags were stripped from the email body field so that classifiers operate on linguistic content rather than markup code. Date formats were standardized to a more human-readable format to ensure consistency and ease of understanding during the annotation phase. No additional stop-word removal, stemming, or lemmatization was performed at the dataset level; such transformations were instead applied within individual model pipelines to allow fair comparison across different feature-extraction strategies.

3.5 Annotation and Agreement

Although the dataset includes pre-existing labels from the original corpora, a secondary annotation pass was conducted to verify label quality and familiarize the evaluation team with the underlying data. The first 480 samples of the dataset were selected for annotation and split equally among four annotators. The annotators then swapped sections and independently re-labeled the first 40 samples of their new partition, producing 160 double-annotated instances. Each sample took approximately 45 seconds to annotate. Inter-annotator agreement was measured using Cohen's Kappa on the 160 double-annotated data points, yielding $\kappa = 0.9$, indicating excellent agreement. This high agreement is partly attributable to the technical background of the team, which made it easier to successfully recognize common phishing indicators such as suspicious URLs, urgent language, and impersonated sender-domains.

3.6 Dataset Partitioning

The 15,000-instance corpus was partitioned into three non-overlapping subsets to support rigorous evaluation and mitigate overfitting: a training set comprising 70% of the data (10,500 emails), a validation set comprising 15% (2,250 emails) for hyperparameter tuning, and a held-out test set comprising the remaining 15% (2,250 emails) whose labels are withheld during development.

3.7 Sensitive Content Disclaimer

Because the dataset contains real-world emails from public archives, evaluators and potential users may encounter content related to financial fraud, fake legal threats, and adult-oriented spam. Some emails may also contain personally identifiable information such as names, email addresses, and phone numbers of individuals. All data is used strictly for research and classification purposes in compliance with the terms of service of the Figshare platform.

4 Features

4.1 Model 1: RoBERTa and TF-IDF Ensemble

To capture both semantic meaning and structural metadata, a single text input was created for each email by concatenating the sender's domain, the subject line, and the email body. The subject line was included twice in this string to increase its weighting in the text representation, which makes sense because subjects often contain the strongest and most immediate indicators of phishing attempts (such as false urgency or administrative threats).

Two distinct feature representations were used to vary the experiments. For the Logistic Regression branch, the text was transformed using TF-IDF on both word n-grams (unigrams to trigrams) and character n-grams (3 to 5 characters). This static sparse representation captures specific phrases and unusual spellings. In addition to the text features, seven numeric features were engineered: the character lengths of the subject and body, the counts of URLs, exclamation marks, dollar signs, a count of urgent keywords (such as "password", "suspended", and "verify"), and a binary flag for the presence of URLs. These features were explicitly added because phishing heavily relies on malicious links, aggressive exclamation formatting, and fabricated urgency to deceive users. For the Transformer branch, rather than static representations, the model utilized dynamic contextual word embeddings. The engineered text was encoded using the `roberta-base` tokenizer, and the embeddings were learned entirely as part of the model, specifically truncated to a maximum length of 512 tokens. Feature selection was also naturally performed, as the TF-IDF vocabulary was capped at the most frequent 80,000 word and 30,000 character n-grams.

4.2 Model 2: Random Forest Classifier

For the Random Forest Classifier, the text input was created by concatenating the *subject* and *body* fields from the dataset, separated by a space. Initial tokens were generated via lowercasing and space separation. These tokens underwent further preprocessing to remove punctuation and stopwords, and each token was subsequently stemmed using the `nltk PorterStemmer` before being rejoined. The resulting text was then passed through a `CountVectorizer` for the bag-of-words vectorization required by the classifier.

The *subject* and *body* fields were concatenated to ensure the model could access all primary plaintext data within an email, making it robustly tokenizable without overly complex preprocessing. This approach also provides a larger contextual window to improve model accuracy. Given the bag-of-words representation, this functions as a form of feature engineering where the frequency or existence of specific tokens (such as language commonly used to advertise explicit content) can be learned by the model to classify the contents as phishing.

Finally, case normalization, stop-word removal, and stemming were implemented to drastically reduce the vocabulary size while maintaining the dataset's key distinguishing features.

4.3 Model 3: RoBERTa Base

Because this approach utilizes a fine-tuned architecture, representation learning was prioritized to take full advantage of the pretrained attention transformer. To effectively utilize this capacity, the text input was formatted as follows:

Listing 1: Input format for the RoBERTa model.

```
[SUBJECT] <{Subject content}>
[BODY] <{Body content}>
[SENDER] <{Sender content}>
```

This structured format allowed RoBERTa's contextualized embeddings to distinctly separate *subject*, *body*, and *sender* information during processing. The specific concatenation order was chosen so that the primary email contents were prioritized during tokenization prior to the sender identity. This ordering helps prevent legitimate but infrequent communicators from being incorrectly flagged as phishing.

4.4 Model 4: ELECTRA Base

Similarly, because ELECTRA is a fine-tuned architecture, representation learning was prioritized to leverage the pretrained attention transformer. To utilize this capacity natively, the text input was formatted as follows:

Listing 2: Input format for the ELECTRA model.

```
[SENDER] <{Sender content}>
[SUBJECT] <{Subject content}>
[BODY] <{Body content}>
```

This layout allowed ELECTRA's contextualized embeddings to distinctly separate *sender*, *subject*, and *body* information. The specific order was selected because it mirrors the natural flow of information in a standard email. Furthermore, because ELECTRA operates as a discriminative model, providing the sender identity first allows it to aggressively prioritize address identification when determining if an email originates from a phishing source.

5 Implementation

Describe your model and implementation here. Refer to item 4. This may take around a page.

5.1 Model 1: RoBERTa and TF-IDF Ensemble

This implementation uses a probability-based ensemble approach that combines a traditional machine learning classifier with a fine-tuned Transformer. Several versions of the model were run independently on the data, effectively serving as an ablation study to see how each standalone framework performed before the ensemble blending.

The first component is a Logistic Regression model trained on the combined TF-IDF and numeric features. The model was trained to minimize the log-loss (cross-entropy) using the L-BFGS optimization technique, with the regularization parameter set to $C = 5.0$.

The second component is a `roberta-base` sequence classification model. Optimization was performed using AdamW to minimize the standard cross-entropy loss function. This model was fine-tuned for 5 epochs with a batch size of 16, a learning rate of 1×10^{-5} with a 15% warmup ratio, and a weight decay of 0.01. The best model checkpoint was saved based on the validation F1-score.

The final classifier blends the predicted probabilities from both models. By testing different blend weights on the validation set, an optimal alpha of 0.55 was found, meaning RoBERTa's prediction contributed slightly more (55%) than the Logistic Regression (45%) to the final output. The classification threshold was also tuned from the default 0.5 to 0.46, maximizing the overall F1-score.

5.2 Model 3: RoBERTa Base

The RoBERTa Base model was fine-tuned for binary sequence classification using the Hugging Face `transformers` library. The pretrained `roberta-base` checkpoint was loaded via `AutoModelForSequenceClassification` with `num_labels=2`, and the corresponding tokenizer was used to encode all inputs with max-length padding and truncation to 512 tokens.

Training was performed using the `Trainer` API with the following hyperparameters: a learning rate of 2×10^{-5}, a per-device batch size of 8 with gradient accumulation over 4 steps (yielding an effective batch size of 32), mixed-precision training (`fp16=True`), weight decay of 0.01, and 5 training epochs. Model checkpoints were saved after each epoch, and the best checkpoint was automatically restored at the end of training based on the validation F1-score.

The loss function used was the standard cross-entropy loss supplied internally by the `Trainer` API for classification heads. The AdamW optimizer (the default for Hugging Face Trainer) was used throughout training. No additional learning-rate scheduler warmup was specified beyond the library defaults.

6 Results and Evaluation

What are your results? What did you implement and what did your classmates implement? What are your baselines? Refer to items 5 and 6. This may take around 0.5-1 pages.

The evaluation strategy partitioned the data into a 70% training split (10,500 samples), a 15% validation split (2,250 samples), and a 15% held-out test split (2,250 samples). To prevent label bias, the class distributions were maintained at a perfectly balanced 50/50 split between phishing and legitimate emails across all dataset partitions. The F1-score was used as the primary evaluation metric because it provides a reliable and unskewed balance between precision and recall. In the context of phishing detection, both metric extremes carry severe consequences: low precision (high false

positives) results in legitimate, potentially critical emails being wrongfully filtered into a spam folder, while low recall (high false negatives) allows dangerous security threats to reach the user's inbox. Therefore, optimizing for the harmonic mean via the F1-score ensures that a model does not aggressively sacrifice one constraint for the other. Accuracy, precision, and recall were also tracked to provide a comprehensive assessment across all implementations.

For comparison, two general baselines were provided by the Codabench benchmark dataset:

- **Majority Vote Baseline:** F1 = 0.6667, Accuracy = 0.5000, Precision = 0.5000, Recall = 1.0000.
- **Logistic Regression with Bag-of-Words (Benchmark):** F1 = 0.8196, Accuracy = 0.8116, Precision = 0.7861, Recall = 0.8560.

6.1 Model 1: RoBERTa and TF-IDF Ensemble

Final test evaluations were obtained by submitting the model's predictions to the Codabench evaluation system. On the test set, the ensemble model achieved an F1-score of 0.9844, an accuracy of 0.9844, a precision of 0.9884, and a recall of 0.9804. This demonstrates that the model successfully generalizes to unseen data, effortlessly outperforming both the provided majority vote baseline and the primary test benchmark.

6.2 Model 3: RoBERTa-Base

The Roberta-Base model achieved a test F1-Score of 0.9852 with a precision of 0.9893 and recall of 0.9849. This shows that the model beats the general baselines and is actually an improvement on trivial methods of prediction. This means that the table is actually worth it.

7 Error Analysis

What kind of errors are your models making? This doesn't have to be super complicated. Confusion matrices are great. Qualitative observations are also great. This means looking at many cases where your model is wrong (wrong class or high error) and looking for patterns yourself. What about the text might be giving it difficulty? Qualitative analysis is underappreciated and can often give you great insight into issues with your model. Examples are useful here. Refer to item 7. This may be around a half page.

7.1 Model 1: RoBERTa and TF-IDF Ensemble

Systematic examination of the evaluation metrics reveals distinct patterns in the errors the model makes. Specifically, the metrics show that the model is extremely good at classifying legitimate emails accurately without falsely flagging them, as indicated by the high precision (0.9884). It rarely produces false positives, which is critical for establishing real-world trust in a spam filter where missing important emails is highly disruptive.

However, the slightly lower recall (0.9804) indicates that the model is comparatively worse at catching highly disguised threats, frequently resulting in false negatives. When comparing the performance of the components, the errors predominantly occur when sophisticated phishing attempts completely omit obvious spam keywords and mimic casual conversational tones flawlessly. Because the model relies substantially on engineered text patterns, these benign-looking emails fail to trigger the classifier. To specifically address these issues in future iterations, we recommend integrating external reputation-checking systems, such as automated URL domain cross-referencing or sender authentication parsing, rather than relying exclusively on the explicit text content.

7.2 Model 3: RoBERTa-Base

8 Conclusions

Summarize your work briefly. Reflect on your implementation and areas for improvement. What do you suggest people work on next (mentioned also in item 7)? This should likely be less than 0.25 pages.

9 Template Notes

You can remove this section or comment it out, as it only contains instructions for how to use this template. You may use subsections in your document as you find appropriate.

9.1 Tables and figures

See Table 2 for an example of a table and its caption. See Figure 1 for an example of a figure and its caption.

9.2 Citations

Table 2 shows the syntax supported by the style files. We encourage you to use the natbib styles. You can use the command `\citet` (cite in text) to

Output	natbib command	ACL only command
(?)	\citep	
?	\citealp	
?	\citet	
(?)	\citeyearpar	
?'s (?)		\citeposs

Table 2: Citation commands supported by the style file.

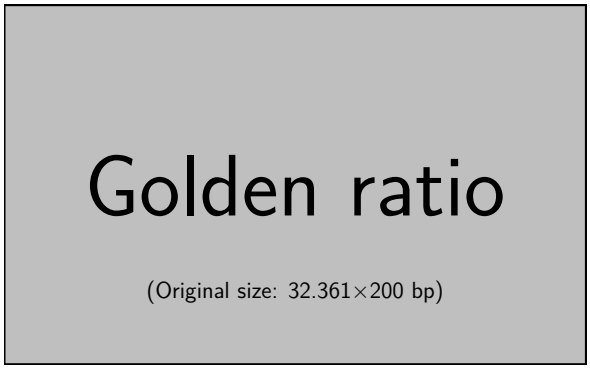

Figure 1: A figure with a caption that runs for more than one line. Example image is usually available through the `mwe` package without even mentioning it in the preamble.

get "author (year)" citations, like this citation to a paper by **?**. You can use the command `\citep` (cite in parentheses) to get "(author, year)" citations (**?**). You can use the command `\citealp` (alternative cite without parentheses) to get "author, year" citations, which is useful for using citations within parentheses (e.g. **?**).

9.3 References

Many websites where you can find academic papers also allow you to export a bib file for citation or bib formatted entry. Copy this into the `custom.bib` and you will be able to cite the paper in the LaTeX. You can remove the example entries.

9.4 Equations

An example equation is shown below:

$$A = \pi r^2 \tag{1}$$

Labels for equation numbers, sections, subsections, figures and tables are all defined with the `\label{label}` command and cross references to them are made with the `\ref{label}` command. This an example cross-reference to Equation 1. You can also write equations inline, like this: $A = \pi r^2$.

Team Contributions

Write in this section a few sentences describing the contributions of each team member. What did each member work on? Refer to item 8.

References

Saeed Abu-Nimeh, Dario Nappa, Xinlei Wang, and Suku Nair. 2007. A comparison of machine learning techniques for phishing detection. In *Proceedings of the Anti-Phishing Working Groups 2nd Annual eCrime Researchers Summit (eCrime '07)*, pages 60–69. ACM.

Areej Abdullah Alhogail and Afrah Alsabih. 2021. Applying machine learning and natural language processing to detect phishing email. *Computers & Security*, 110:102414.

Ammar Almomani, Brij B. Gupta, Samer Atawneh, and Eman Almomani. 2013. A survey of phishing email filtering techniques. *IEEE Communications Surveys & Tutorials*, 15(4):2070–2090.

Ram B. Basnet, Srinivas Mukkamala, and Andrew H. Sung. 2008. Detection of phishing attacks: A machine learning approach. In *Soft Computing Applications in Industry*, volume 226 of *Studies in Fuzziness and Soft Computing*, pages 373–383. Springer, Berlin, Heidelberg.

Arifa I. Champa, Md. Fazle Rabbi, and Minhaz F. Zibran. 2024. Curated datasets and feature analysis for phishing email detection with machine learning. In *Proceedings of the 3rd IEEE International Conference on Computing and Machine Intelligence (ICMI)*, pages 1–7.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Ian Fette, Norman Sadeh, and Anthony Tomasic. 2007. Learning to detect phishing emails. In *Proceedings of the 16th International Conference on World Wide*

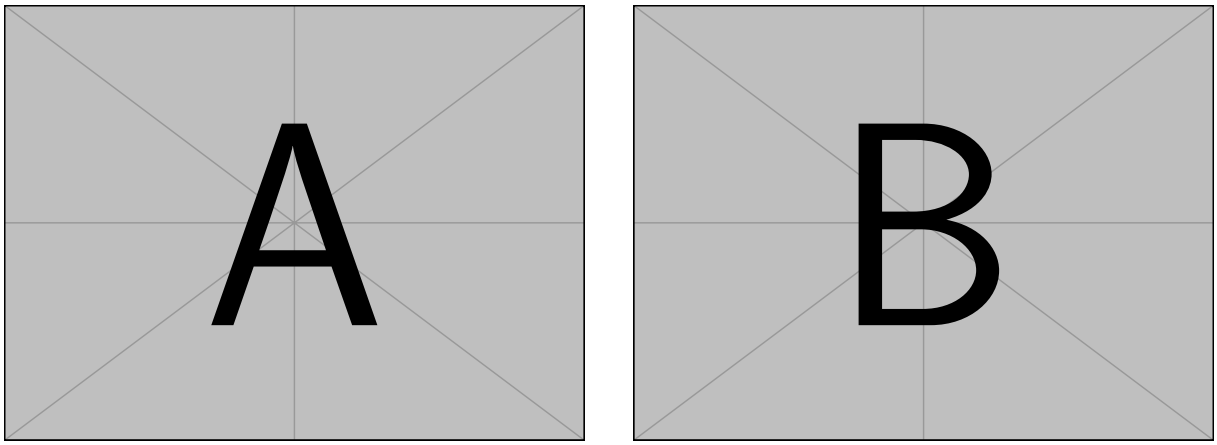

Figure 2: A minimal working example to demonstrate how to place two images side-by-side.

Web (WWW '07), pages 649–656, Banff, Alberta, Canada.